9주차 예습과제-자연어처리와 컴퓨터비전 심층학습 10장

10장 - 비전 트랜스포머 (ViT)

> 💡 이미지를 **퍼즐 조각(패치)**으로 잘라 **문장의 단어처럼** 한 줄로 세운 뒤, 트랜스포머에 그대로 넣어 분류하는 모델.
> CNN처럼 필터로 한 구역씩 훑는 게 아니라, **셀프 어텐션으로 모든 패치가 서로를 한 번에** 참고한다.

10.1 ViT 개요

1. ViT란?

- 구글(Google)에서 개발 (2020)
- 논문: "An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale"
- 핵심: 이미지를 **자연어 처리 방식처럼** 분류해 보려는 시도 → CNN의 합성곱 계층 대신 트랜스포머의 **셀프 어텐션** 적용

구분	합성곱 모델 (CNN)	ViT
핵심 연산	합성곱 계층	셀프 어텐션
특징 추출	**지역 특징**을 추출해서 분류	**전체 이미지를 한 번에** 처리

> 🧠 CNN은 돋보기로 사진을 한 구역씩 보는 사람, ViT는 사진 전체를 한눈에 훑으며 "이 부분이랑 저 부분이 관련 있네"를 찾는 사람.

2. ViT의 한계 → 후속 모델 등장

- ViT는 이미지 패치가 **왼→오, 위→아래 순서로 한 줄**로 입력됨 → 2차원(가로·세로) 구조를 온전히 반영한다고 할 수 없음
- 해결책: 물체의 **크기나 해상도를 계층적으로** 학습하는 모델 제안

모델	핵심 아이디어	방법
스윈 트랜스포머 (Swin Transformer)	**로컬 윈도우(Local Window)** 활용	각 계층의 어텐션이 되는 패치의 크기와 개수를 다양하게 구성. 셀프 어텐션을 ① 로컬 윈도우 **안** ② 로컬 윈도우 **간** 어텐션으로 수행 → 계층적 학습. **상대적 위치 편향**을 어텐션 값에 반영
CvT (Convolutional Vision Transformer)	합성곱 연산을 ViT에 적용	**저수준 특징**(눈·코·입) + **고수준 특징**(눈·코·입을 포함한 얼굴 전체)을 계층적으로 반영. 어텐션의 **K(키)·V(값)를 축소**해 계산 복잡도 감소

> 🔎 Q·K·V 복습 — **Q(쿼리)** = "내가 찾는 것", **K(키)** = "각자 붙인 이름표", **V(값)** = "실제 내용물". 키·값을 줄이면 비교할 대상이 줄어서 계산이 가벼워짐.

- ViT 계열 모델들은 이미지 분류에 매우 효과적임이 입증됨 → 다른 컴퓨터비전 작업에도 적용 가능

3. 실습 데이터 — FashionMNIST

- 10개의 패션 상품을 분류하는 문제

[그림 10.1 FashionMNIST 데이터 샘플 이미지]

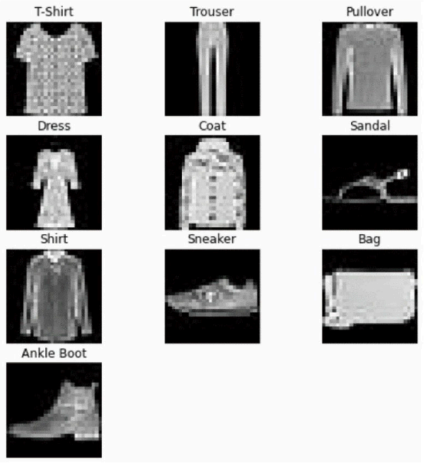


그림 10.1 FashionMNIST 데이터 샘플 이미지

번호	0	1	2	3	4	5	6
클래스	티셔츠	바지	풀오버	드레스	코트	샌들	셔츠

10.2 ViT 구조

1. BERT vs ViT — 입력 만드는 과정만 다르다

- ViT = 트랜스포머 구조 **자체**를 컴퓨터비전에 적용한 **첫 번째** 연구
  - 이전 연구들: 합성곱 신경망에 트랜스포머의 셀프 어텐션 모듈을 "끼워 넣은" 모델이 많았음

[그림 10.2 BERT와 ViT 모델 구조 차이]

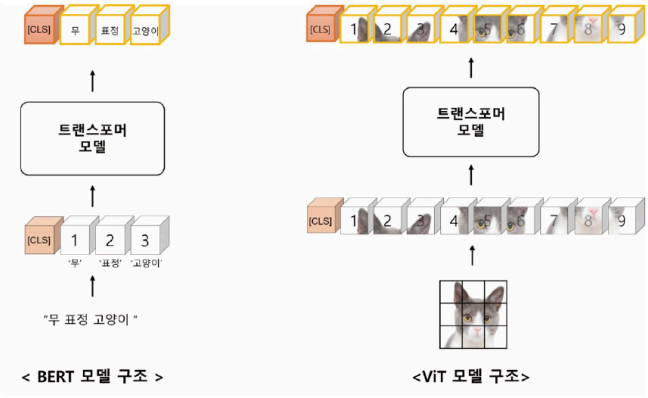


그림 10.2 BERT의 ViT 모델 구조 차이

구분	BERT	ViT
입력	"무 표정 고양이" → 토큰 1, 2, 3	고양이 이미지 → 격자로 나눈 패치 1~9
입력 단위	문장보다 작은 단위의 **토큰**	작은 단위의 **이미지 패치**
입력 순서	순차적	왼→오, 위→아래 시퀀셜 배열로 가정
맨 앞 토큰	[CLS]	[CLS]
몸통	트랜스포머 모델 그대로	트랜스포머 모델 그대로

2. 합성곱 모델 vs ViT 모델 비교

- 목적은 같음: 이미지 특징을 잘 표현하는 **임베딩** 만들기 → 과정이 다름

[그림 10.3 합성곱 신경망과 ViT 계층의 차이]

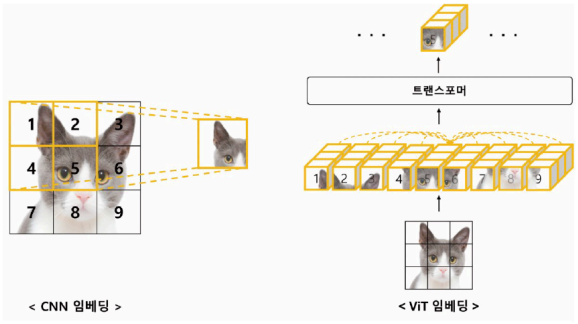


그림 10.3 합성곱 신경망과 ViT 계층의 차이

구분	CNN 임베딩	ViT 임베딩
학습 방식	이미지 패치 중 **일부만 선택**해 학습	이미지를 패치로 나눠 **패치 간 상관관계** 학습
예시 (고양이 오른쪽 눈)	패치 **1, 2, 4, 5만** 학습 / 3, 6, 7, 8, 9는 관여 X	셀프 어텐션으로 **모든 패치(1~9)가** 학습에 관여
결과	지역 특징 → 조합해 전체 특징 추출	모든 패치가 서로에게 주는 영향을 고려 → 높은 수준의 이미지 표현

3. ViT의 장단점

구분	내용
✅ 장점 1	CNN은 좁은 **수용 영역(Receptive Field, RF)** 때문에 전체 이미지 정보를 표현하려면 **수많은 계층**이 필요 → 트랜스포머는 **어텐션 거리(Attention Distance)**를 계산해 **ViT 레이어 하나로도** 전체 이미지 정보 표현 가능
✅ 장점 2	픽셀 단위로 처리하는 CNN과 달리 **패치 단위** 처리 → 더 작은 모델로도 높은 성능
❌ 단점 1	입력 이미지 크기가 **고정** → 크기가 다른 이미지는 크기를 맞추는 전처리 필요
❌ 단점 2	CNN은 이미지의 공간적 위치 정보를 고려하지만 ViT는 패치 간 **상대적 위치 정보만** 고려 → 이미지 변환에 취약할 수 있음

> 🔎 **수용 영역** = 뉴런 하나가 원본 이미지에서 "보고 있는 범위". CNN은 층 하나가 3×3 정도만 보니까, 전체를 보려면 층을 계속 쌓아야 함. ViT는 첫 층부터 모든 패치를 볼 수 있음.

4. ViT의 귀납적 편향

- **귀납적 편향(Inductive Bias)** = 일반화 성능 향상을 위한 모델의 **가정(Assumption)**

[표 10.1 딥러닝 모델의 귀납적 편향]

딥러닝 모델	귀납적 편향	가정	강한 데이터
합성곱 신경망 (CNN)	**지역적(Local) 편향**	공간적 관계가 중요 → 지역 특징 강조	이미지
순환 신경망 (RNN)	**순차적(Sequential) 편향**	이전 시간의 상태 + 현재 입력 → 다음 상태 예측	시계열
ViT	**귀납적 편향이 거의 없음**	입력 데이터의 다양한 쿼리·키·값 임베딩 형태로 일반화된 관계 학습	대용량 데이터

- 가정이 **올바르면** → 더 적은 데이터로 높은 일반화 성능
- 가정이 **너무 강하면** → 다른 유형의 데이터나 관계를 표현하기 어려움 → 귀납적 편향이 약한 모델을 선호하게 됨

5. ViT 모델 구조

- ViT = **패치 임베딩(Patch Embedding)** + **인코더(Encoder) 계층**
  - 패치 임베딩: 입력 이미지를 일정한 크기의 패치로 나눈 뒤 각 패치를 **벡터 형태로 변환**
  - 인코더 계층: 각 패치와의 **관계를 학습**
- 이후 분류·회귀 등 작업에 맞는 출력값으로 변환해 사용

[그림 10.4 ViT 모델 구조]

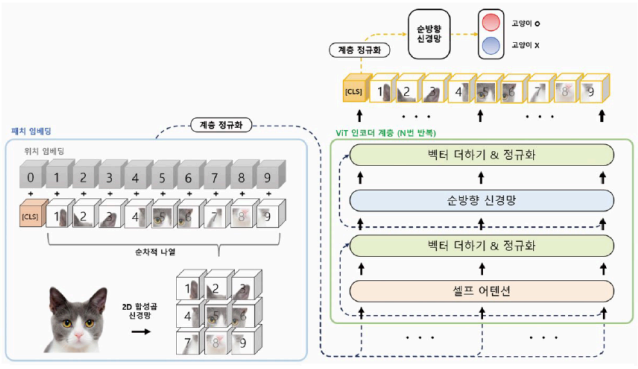


그림 10.4 ViT 모델 구조

단계	구성	하는 일
① 패치 생성	고양이 이미지 → **2D 합성곱 신경망** → 패치 1~9	이미지를 조각으로 자름
② 순차적 나열	[CLS] + 패치 1~9	한 줄로 세우고 맨 앞에 분류 토큰
③ 위치 임베딩	0, 1, 2, ..., 9를 각 패치에 **더함 (+)**	자리 번호표 붙이기
④ 계층 정규화	–	패치 임베딩 완성
⑤ ViT 인코더 계층 (**N번 반복**)	셀프 어텐션 → 벡터 더하기 & 정규화 → 순방향 신경망 → 벡터 더하기 & 정규화	패치 간 관계 학습
⑥ 분류	[CLS] 출력 → 계층 정규화 → 순방향 신경망	고양이 O / 고양이 X

6. 패치 임베딩 (Patch Embedding)

- 정의: 입력 이미지를 **작은 패치로 분할**하는 과정
- 과정
  1. 이미지 크기를 맞추는 전처리 (예: 640×480 → **224×224** 정사각형)
  2. 전체 이미지를 패치 크기로 분할해 **시퀀셜 배열** 생성 → 이때 **합성곱 신경망의 계층**을 활용

[그림 10.5 이미지 패치 생성 과정]

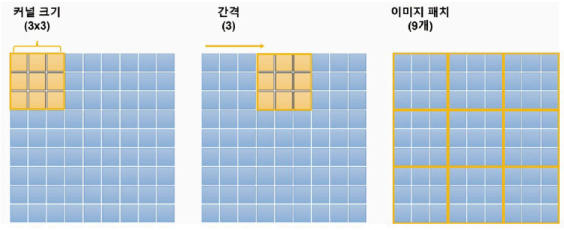


그림 10.5 이미지 패치 생성 과정

하이퍼파라미터	의미	그림 예시
커널 크기 (Kernel Size)	패치의 크기	3×3
간격 (Stride)	패치가 이동하는 폭	3
결과	9×9 이미지 →	**9(3×3)개** 이미지 패치

> 🧠 커널 = 쿠키 틀 크기, 간격 = 쿠키 틀을 한 번에 옮기는 거리. 틀 크기만큼 옮기면 겹치지 않는 쿠키가 딱딱 찍힘.

수식 10.1 패치 계산 과정 (가로 또는 세로 패치 개수)

$$patch\ size = \frac{image\ size - kernel\ size}{stride} = \frac{9-3}{3} + 1 = 3$$

- 기호 풀이: (이미지 길이 – 틀 크기) = 틀이 움직일 수 있는 거리 → ÷ 간격 = 몇 번 옮길 수 있나 → +1 = 처음 놓는 한 번

이미지	커널	간격	가로(세로) 패치	전체 패치
9×9	3×3	3	(9−3)/3+1 = 3	3×3 = **9개** (안 겹침)
9×9	3×3	1	(9−3)/1+1 = 7	7×7 = **49개** (겹침)
224×224 (실습)	16×16	16	(224−16)/16+1 = 14	14×14 = **196개**

- 이어지는 단계

단계	내용
순서 배열	분할된 패치를 왼→오, 위→아래 순으로 배열
분류 토큰 추가	배열 가장 왼쪽에 **[CLS]** (Special Classification Token) 추가 → 전체 이미지를 대표하는 벡터로 특정 문제 예측에 사용
위치 임베딩 (Position Embedding)	패치의 위치 정보를 임베딩 벡터로 변환 → 기존 이미지 패치 벡터들과 **더함** → 인접 패치 간 관계 학습
계층 정규화	마지막에 적용 → 패치 임베딩 완성

- 이미지를 작은 패치로 나눠 처리 → **GPU 메모리 한계 극복**, 더 큰 이미지 처리 가능

> 🧠 [CLS]는 반장. 패치들의 이야기를 다 듣고(어텐션) 마지막에 반 대표로 "이건 고양이입니다"라고 발표함.

7. 인코더 계층 (Encoder)

- 패치 벡터들을 인코더 계층에 입력 → 다양한 쿼리·키·값 임베딩의 관계를 학습
  - **7장 트랜스포머의 멀티 헤드 어텐션 학습 과정과 동일**
- N개의 인코더 레이어를 반복 적용 → 마지막 레이어에서 **분류 토큰(특별한 패치)의 특징 벡터** 추출
  - 이 분류 토큰 벡터 = 이미지 데이터를 잘 표현하는 특징 벡터로 간주 → 다양한 이미지 분류·검색 문제에 사용
- 예: 고양이인지 아닌지 구분
  - **시퀀셜 산출물(Sequential Output)**(= 마지막 인코더에서 나온 패치들의 특징 벡터)을 모두 쓰지 **않음**
  - **분류 토큰 벡터만** → 순방향 신경망(또는 완전 연결 계층)에 연결 → 고양이 O/X 분류

10.3 ViT — 허깅페이스 실습 (모델 실습)

> 💡 FashionMNIST를 클래스별로 균등하게 줄여서 → ViT 입력 규격(224×224, 3채널, 정규화)에 맞게 전처리 → collate_fn으로 **딕셔너리** 배치를 만들고 → 사전 학습된 ViT의 **분류기를 10개로 교체**해 미세 조정.
> 패치 임베딩 shape이 `[32, 3, 224, 224] → [32, 196, 768] → [32, 197, 768]`로 바뀌는 과정 확인.

Overall Steps

단계	코드	하는 일
1	예제 10.1	FashionMNIST 다운로드 → 클래스별 균등 샘플링 (학습 10,000 / 테스트 1,000)
2	예제 10.2	AutoImageProcessor로 사전 학습 모델의 전처리 설정 불러와 transform 정의
3	예제 10.3	collate_fn(collator)으로 딕셔너리 배치 만드는 데이터로더
4	예제 10.4	사전 학습된 ViT 불러오기 + 분류기 10개로 교체
5	예제 10.5	패치 임베딩 구조·shape 확인
6	예제 10.6	TrainingArguments로 하이퍼파라미터 설정
7	예제 10.7	매크로 평균 F1 점수 평가 함수
8	예제 10.8	Trainer로 학습 (다음 파트)

1. 데이터 불러오기 & 균등 샘플링 (예제 10.1)

- 데이터: `torchvision.datasets.FashionMNIST(root, download=True, train=True/False)`
  - `root` : 데이터셋이 저장될 디렉터리 위치
  - `download=True` : 데이터셋이 없으면 다운로드
  - `train` : True → 훈련용, False → 테스트용
- 원본: 10개 클래스, 훈련 **60,000** / 테스트 **10,000** → 간단한 실습을 위해 줄임

속성	의미
`classes`	클래스 목록 ['T-shirt/top', 'Trouser', ..., 'Ankle boot']
`class_to_idx`	클래스 이름 → ID 매핑 {'T-shirt/top': 0, ..., 'Ankle boot': 9}

subset_sampler(dataset, classes, max_len) — 서브 샘플링 함수

단계	핵심 키워드	하는 일
①	`defaultdict(list)`	기본값이 있는 딕셔너리 → 클래스별로 데이터 **색인 값(인덱스)** 모으기
②	`enumerate(dataset.train_labels)`	(인덱스, 레이블)을 돌면서 `target_idx[int(label)].append(idx)`
③	`[:max_len]`	클래스마다 앞에서 max_len개만
④	`chain.from_iterable`	클래스별 리스트를 **1차원 리스트로 펴기**
⑤	`Subset(dataset, indices)`	해당 색인 값의 데이터만 추출한 서브셋 반환

데이터	max_len	결과
subset_train_dataset	1000	1,000 × 10 = **10,000개**
subset_test_dataset	100	100 × 10 = **1,000개**

- 출력 `train_dataset[0]` → `(<PIL.Image mode=L size=28x28>, 9)`
  - PIL 이미지 + 클래스 번호, `mode=L` = **흑백 1채널**, 28×28, 9 = 앵클 부츠

> 🧠 클래스마다 똑같은 개수를 뽑는 이유 — 한 반에서만 학생을 많이 뽑으면 그 반 의견만 반영되듯, 클래스 불균형이 생기면 모델이 많은 쪽만 잘 맞히게 됨.

2. 이미지 전처리 (예제 10.2)

- 모델 학습엔 `PIL.Image`가 아닌 `Tensor` 형식 필요 → 전처리
- `AutoImageProcessor.from_pretrained("google/vit-base-patch16-224-in21k")`
  - 이미지 프로세서 클래스 = **사전 학습된 ViT 모델의 전처리 설정**(자르기 crop, 크기 조정 resize 등)을 그대로 가져옴

출력	값
size	{'height': 224, 'width': 224}
mean	[0.5, 0.5, 0.5]
std	[0.5, 0.5, 0.5]

transforms.Compose 구성

순서	변환	하는 일	왜?
1	`ToTensor()`	PIL → Tensor (값 0~1)	모델 입력 형식
2	`Resize(size=(height, width))`	28×28 → **224×224**	ViT 입력 크기에 맞춤
3	`Lambda(lambda x: torch.cat([x, x, x], 0))`	**단일 채널을 복제**해 다중(3) 채널 이미지로	사전 학습 모델은 컬러 3채널 입력
4	`Normalize(mean, std)`	(값 − 0.5) / 0.5 → **0~1** → **−1~1**	사전 학습 때와 같은 값 범위

사용 모델 이름 해석: `google/vit-base-patch16-224-in21k`

부분	의미
`vit-base`	ViT 기본 크기 (임베딩 차원 768)
`patch16`	패치 크기 **16×16**
`224`	입력 이미지 **224×224**
`in21k`	**ImageNet-21k**로 사전 학습 (약 1,400만 개 이미지, 21,843개 클래스)

3. 데이터로더 & 집합 함수 (예제 10.3)

- ViT 모델은 Tensor 하나가 아니라 **딕셔너리**를 입력으로 받음
  - 입력 형태: `{"pixel_values": pixel_values, "labels": labels}`

- 그래서 데이터로더에 **집합 함수(collate_fn)** 적용 → 미니 배치의 샘플 목록을 병합해 ViT 구조에 맞는 형태로 변환

collator(data, transform)

단계	핵심 키워드	하는 일
①	`zip(*data)`	[(이미지, 레이블), ...] → (이미지들), (레이블들) 분리
②	`torch.stack([transform(image) …])`	각 이미지를 변환 후 **한 덩어리로 쌓기**
③	`torch.tensor([label …])`	레이블들을 텐서로
④	`return {"pixel_values": …, "labels": …}`	딕셔너리로 반환

DataLoader 설정

이터레이터	데이터	batch_size	shuffle	collate_fn	drop_last
train_dataloader	subset_train_dataset	**32**	True	`lambda x: collator(x, transform)`	True
valid_dataloader	subset_test_dataset	**4**	True	`lambda x: collator(x, transform)`	True

- `next(iter(train_dataloader))` 로 첫 배치 확인

키	shape	의미
pixel_values	`torch.Size([32, 3, 224, 224])`	(배치 크기, 채널 수, 이미지 높이, 이미지 너비)
labels	`torch.Size([32])`	클래스 색인 값 32개

> 🧠 collate_fn은 택배 포장 담당. 낱개로 온 물건(이미지·레이블)을 상자 하나에 담고 "pixel_values / labels" 송장을 붙여서 모델에게 보냄.
> 🔎 `drop_last=True` = 마지막에 배치 크기보다 적게 남은 자투리는 버림.

4. 사전 학습된 ViT 불러오기 (예제 10.4)

- **ViT 이미지 분류 모델 클래스** `ViTForImageClassification.from_pretrained(...)`
  - 사전 학습된 모델 = 앞의 이미지 프로세서와 **동일한 모델** (`vit-base-patch16-224-in21k`)
- 원래 **21,841개** 분류 카테고리로 학습된 모델 → 미세 조정 필요
  - 8장 방식으로 직접 바꿔도 되지만, 이 클래스는 **매개변수만으로** 미세 조정 가능

매개변수	값	의미
`num_labels`	`len(classes)` = 10	레이블 개수
`id2label`	`{idx: label …}`	ID → 레이블 (9 → 'Ankle boot')
`label2id`	`class_to_idx`	레이블 → ID
`ignore_mismatched_sizes`	True	분류기 크기(21k → 10)가 달라도 에러 없이 새로 만들어 교체

- 출력 `model.classifier` → `Linear(in_features=768, out_features=10, bias=True)`
  - 분류기 출력 차원 크기(out_features)가 **10개**로 바뀜 → **768차원 [CLS] 벡터 → 10개 클래스 점수**

5. 패치 임베딩 확인 (예제 10.5)

- 출력 `model.vit.embeddings`

구성	설정
patch_embeddings → projection	`Conv2d(3, 768, kernel_size=(16, 16), stride=(16, 16))`
dropout	`Dropout(p=0.0)`

함수	하는 일
패치 임베딩 함수 `model.vit.embeddings.patch_embeddings`	이미지 → 패치 벡터 (자르기 + 벡터화만)
임베딩 함수 `model.vit.embeddings`	패치 앞에 **[CLS] 토큰**을 붙임 (+ 위치 임베딩)

⭐ shape 변화 따라가기

단계	shape	의미
image	`[32, 3, 224, 224]`	32장, 3채널, 224×224
Conv2d (16×16 커널, 간격 16)	`[32, 768, 14, 14]`	(224−16)/16+1 = **14** → 14×14 텐서
일렬로 나열 → patch embeddings	`[32, 196, 768]`	14×14 = **196개** 패치, 각 768차원
[CLS] + patch embeddings	`[32, 197, 768]`	[CLS] 1개 + 196 = **197개** 벡터

> 🔎 Conv2d를 "자르기 도구"로 쓰는 트릭 — 커널 = 간격 = 16이면 필터가 겹치지 않고 16×16 조각마다 딱 한 번씩 찍힘 → 조각 하나가 곧바로 768차원 벡터 하나가 됨. 출력 채널 768 = 임베딩 차원.
> ⚠️ 교재 본문 수식 "(224−4)/4+1 = 14"는 오타 → **(224−16)/16+1 = 14**가 맞음.

- ViT 모델은 7장 트랜스포머와 유사하지만, **입력 데이터를 일정한 크기의 패치로 분할** → **패치마다 특성 추출**하는 과정이 추가됨 → 그래서 **패치 임베딩(patch_embeddings)**이 추가된 것

6. 하이퍼파라미터 설정 (예제 10.6)

- **학습 매개변수 클래스** `TrainingArguments` → 모델 학습에 필요한 다양한 인자들을 저장·관리
  - 하이퍼파라미터 관리, 모델 평가, 분산 학습, 허깅 페이스 허브 연동, 로그 추적 등
  - **80여 개**의 매개변수로 다양한 기능 설정 가능
- 이후 트랜스포머스 라이브러리의 **훈련자 클래스(Trainer)**로 학습

표 10.2 학습 매개변수 클래스 (+ 이번 예제 값)

매개변수	의미	예제 값
`output_dir`	체크포인트 저장 경로	"../models/ViT-FashionMNIST"
`save_strategy`	체크포인트 저장 간격 설정 (no: 저장 안 함 / steps: 스텝마다 / epoch: 에폭마다)	"epoch"
`save_steps`	save_strategy="steps"일 때 스텝 간격 설정	–
`save_total_limit`	체크포인트 최대 저장 개수	–
`evaluation_strategy`	체크포인트 평가 간격 설정 (no / steps / epoch)	"epoch"
`learning_rate`	초기 학습률	1e-5
`per_device_train_batch_size`	학습 배치 크기	16
`per_device_eval_batch_size`	평가 배치 크기	16
`num_train_epochs`	학습 반복 수	3
`weight_decay`	가중치 감쇠	0.001
`load_best_model_at_end`	모델 불러오기 시 최상의 모델 선택 여부	True
`metric_for_best_model`	최상의 모델 선정 기준이 되는 평가 방식 (accuracy: 정확도 / precision: 정밀도 / f1: F1 점수 / mae: 평균 절대 오차 / mse: 평균 제곱 오차 / rmse: 평균 제곱근 편차)	"f1"
`logging_dir`	로그 저장 폴더	"logs"
`logging_steps`	로그 출력 간격	125
`seed`	학습 시작 시 설정되는 무작위 시드	7
`fp16`	Mixed precision 사용 여부	–
`remove_unused_columns`	모델이 안 쓰는 열 자동 삭제 여부	False

> 🔎 **fp16 (혼합 정밀도)** = 일부 매개변수는 32비트로 유지하고 일부는 **16비트로 변환**해 학습 → 메모리 사용량이 줄어 더 큰 배치 크기 사용 가능.
> 🔎 **weight_decay** = 가중치가 너무 커지지 않게 벌점을 주는 것 → 과적합 방지.

7. 성능 평가 지표 — 매크로 평균 F1 (예제 10.7)

- 성능 평가 방법: **매크로 평균 F1 점수(Macro Average F1-Score)**
  - 다중 클래스 문제 → 모든 클래스에 대한 F1 점수의 평균으로 전체 모델 성능 측정

compute_metrics(eval_pred)

단계	핵심 키워드	하는 일
①	`evaluate.load("f1")`	**메트릭 불러오기 함수** — 허깅 페이스 평가 라이브러리 (accuracy, precision, f1 등 제공)
②	`predictions, labels = eval_pred`	모델이 예측한 결괏값 / 실젯값 전달받음
③	`np.argmax(predictions, axis=1)`	모든 클래스 예측값 중 **가장 확률이 높은 값** 선택
④	`metric.compute(predictions, references=labels, average="macro")`	예측값·실젯값을 **메트릭 연산** 메서드에 전달, 평균 방식 = macro

> 🔎 **F1 점수** = 정밀도("맞다고 한 것 중 진짜 맞은 비율")와 재현율("진짜 정답 중 찾아낸 비율")의 균형 점수.
> **매크로 평균** = 클래스 10개 각각의 F1을 구한 뒤 **가중치 없이 단순 평균** → 모든 클래스를 1표씩 똑같이 취급.

8. ViT 모델 학습 (예제 10.8)

- 앞의 예제 10.1~10.7 코드를 **한 번에 모은 전체 코드** + `Trainer`
- **훈련자 클래스(Trainer)** = 트랜스포머스 라이브러리의 학습 담당 → 학습 루프·평가·저장을 알아서 해줌

새로 등장한 함수: model_init(classes, class_to_idx)

- `ViTForImageClassification.from_pretrained(...)`로 모델을 만들어 **return**하는 함수
- 훈련자 클래스는 모델을 **함수(model_init) 형태로** 받아서 생성

> 🔎 왜 모델 자체가 아니라 함수로 줄까? Trainer가 필요할 때마다 **새 모델을 깨끗하게 다시 만들 수 있게** 하기 위해 (같은 seed로 재현하거나, 하이퍼파라미터를 바꿔가며 여러 번 학습할 때 유용).

Trainer 매개변수

매개변수	넣은 값	의미
`model_init`	`lambda x: model_init(classes, class_to_idx)`	모델 생성 함수
`args`	`args` (TrainingArguments)	하이퍼파라미터 설정
`train_dataset`	`subset_train_dataset`	학습 데이터
`eval_dataset`	`subset_test_dataset`	평가 데이터
`data_collator`	`lambda x: collator(x, transform)`	집합 함수 (딕셔너리 배치 포장)
`compute_metrics`	`compute_metrics`	평가 지표 (매크로 F1)
`tokenizer`	`image_processor`	이미지 프로세서 전달

- 훈련자 클래스는 **데이터로더를 내부적으로** 수행 → 그래서 DataLoader 대신 **데이터세트 + 집합 함수**를 직접 전달
- **토크나이저(tokenizer)**: 원래는 토큰을 임베딩해 모델 입력으로 쓰도록 토큰 분리 작업을 하는 것 → 여기선 이미지 프로세서를 그 자리에 넣음
- `trainer.train()` → 학습 실행

> ⚠️ 최신 transformers 버전에서는 Trainer의 `tokenizer` 인자가 ** `processing_class`**로 바뀜. 에러 나면 `processing_class=image_processor`로 수정.

학습 결과 저장

- 학습 완료 시 **ViT-FashionMNIST 폴더**에 체크포인트마다 학습 결과, 성능 지표, 하이퍼파라미터 등 저장

표 10.3 ViT 학습 결과

Epoch	Training Loss	Validation Loss	F1-Score
1	0.7062	0.6377	0.8905
2	0.4954	0.4781	0.9166
3	**0.4078**	0.4357	**0.9231**

- 최종: 손실값 0.4078, F1 점수 0.9231 → 학습이 우수하게 진행됨

- 저장된 체크포인트를 불러오려면 → `model_init` 함수의 `pretrained_model_name_or_path`를 `"../models/ViT-FashionMNIST/checkpoint-1875"`처럼 설정

> 🔎 왜 하필 1875? 학습 데이터 10,000개 ÷ 배치 16 = 에폭당 **625 스텝** × 3 에폭 = **1875 스텝**. 체크포인트 이름 = "몇 번째 스텝에서 저장했나".

9. 모델 성능 평가 — 혼동 행렬 (예제 10.9)

- **혼동 행렬(Confusion Matrix)** = "정답이 A인데 모델이 B라고 한 게 몇 개?"를 표로 정리한 것

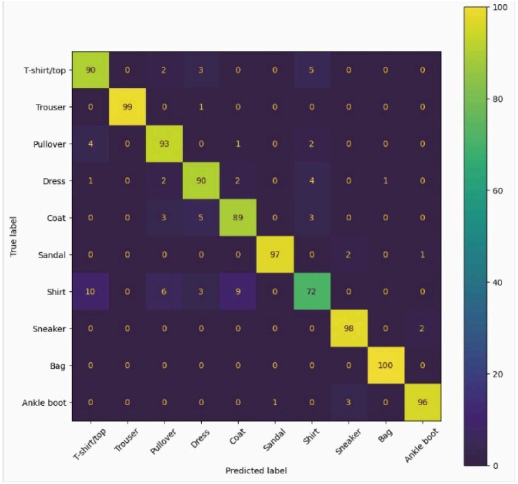
핵심 키워드

코드	하는 일
`trainer.predict(subset_test_dataset)`	테스트 데이터로 예측 → 성능 지표 + 예측값 반환
`outputs.label_ids`	실제 정답 → `y_true`
`outputs.predictions.argmax(1)`	10개 점수 중 가장 높은 클래스 → `y_pred`
`confusion_matrix(y_true, y_pred)`	sklearn으로 혼동 행렬 계산
`ConfusionMatrixDisplay(confusion_matrix=matrix, display_labels=labels)`	행렬을 그림으로
`plt.subplots(figsize=(10, 10))` / `display.plot(xticks_rotation=45, ax=ax)`	그림 크기 / x축 글자 45도 회전

출력 결과 (predict 지표)

지표	값
test_loss	0.4358
test_f1	**0.9232**
test_runtime	8.4356초
test_samples_per_second	29.636
test_steps_per_second	4.283

[혼동 행렬]



- 읽는 법: **세로(True label) = 정답**, **가로(Predicted label) = 모델 예측**, **대각선 = 맞힌 개수** (클래스당 100개)

클래스	맞힘	주로 헷갈린 대상
T-shirt/top	90	Shirt 5, Dress 3
Trouser	99	–
Pullover	93	T-shirt 4
Dress	90	Shirt 4
Coat	89	Dress 5
Sandal	97	Sneaker 2

클래스	맞힘	주로 헷갈린 대상
Shirt	**72** ⚠️	**T-shirt 10, Coat 9, Pullover 6**
Sneaker	98	Ankle boot 2
Bag	100	–
Ankle boot	96	Sneaker 3

- 분류 정확도 **약 92%** (대각선 합 924 / 1000) → 준수한 성능
- 하지만 **셔츠를 티셔츠·코트로 오분류**한 경우가 상대적으로 많음

> 🧠 흑백 28×28 사진에서 셔츠·티셔츠·코트는 사람 눈으로 봐도 비슷하게 생김. 전체 점수(F1)만 보면 이런 약점이 숨는데, 혼동 행렬은 "어디서 틀리는지"를 보여주는 성적표 상세 내역.

- 주요 모델 개선 방법: **하이퍼파라미터 조정, 모델 구조 변경, 전처리 과정 개선, 데이터 증강**
  → 오분류 클래스의 문제점을 해결해 더 높은 정확도 달성 가능

10.4 스윈 트랜스포머 (Swin Transformer)

1. 스윈 트랜스포머란?

- 2021년 발표된 대규모 비전 인식 모델 (마이크로소프트)
- 이미지 분류, 객체 감지, 영상 분할 등에서 강력한 성능 → 객체 탐지·객체 분할 작업의 **백본(backbone, 뼈대) 모델**로 사용

기존 트랜스포머 기반 모델의 문제 → 스윈의 해결

기존 문제	왜 문제?	스윈의 해결
고정된 패치 크기	세밀한 예측이 필요한 **의미론적 분할(Semantic Segmentation)** 같은 작업에 부적합	계층마다 패치 크기·개수를 다르게 → **계층적(hierarchy) 특징 맵**
셀프 어텐션이 이미지 크기에 대해 **2차(Quadratic) 계산 복잡도**	고해상도 이미지 처리 어려움	로컬 윈도우 안에서만 어텐션 → **1차(Linear) 계산 복잡도**

> 🔎 **2차 vs 1차 복잡도** — 패치가 N개일 때, 전체 어텐션은 모든 패치끼리 비교하니 N×N번(=N^2). 이미지가 2배 커지면 계산은 4배. 윈도우 안에서만 비교하면 윈도우 크기가 고정이라 계산이 N에 **비례**만 함 → 2배 커지면 2배.
> 🔎 **의미론적 분할** = 픽셀 하나하나에 "이건 고양이, 이건 배경" 라벨을 붙이는 작업 → 큰 패치 하나로는 세밀하게 못 함.

시프트 윈도우 (Shifted Window)

- 입력 이미지를 일정 크기 패치로 분할 → 패치들을 쌓아 **윈도우** 구성 → **윈도우 영역 안에서만** 셀프 어텐션
- 계층마다 어텐션되는 패치의 크기·개수를 계층적으로 적용 → 높은 해상도·다양한 크기의 객체를 효율적으로 처리

2. ViT와 스윈 트랜스포머 차이

구분	ViT의 한계
패치	획일적인 패치 크기·위치 제약 → 다양한 이미지 크기·종횡비 데이터셋에 적용 어려움
공간 정보	패치 단위 입력 → 이미지 공간 정보가 완전히 보존되지 않을 수 있음
계산량	패치 수 증가 → 학습 데이터 크기·계산량 증가 → 대규모 데이터셋에서 컴퓨팅 자원 많이 사용

- 스윈의 보완
  - **로컬 윈도우(Local Window)** = 입력 이미지 처리에 쓰는 **고정 크기의 작은 윈도우** → 물체 크기·해상도를 계층적으로 학습
  - **시프트 윈도우** = 로컬 윈도우를 이동 → 특징 맵을 이동시켜 다음 계층에 사용 → 패치 위치를 더 유연하게 다룸
  - 결과: ViT보다 **더 높은 정확도 + 더 적은 매개변수**로 더 넓은 범위의 이미지 크기·종횡비 처리

[그림 10.6 ViT와 스윈 트랜스포머의 네트워크 구조]

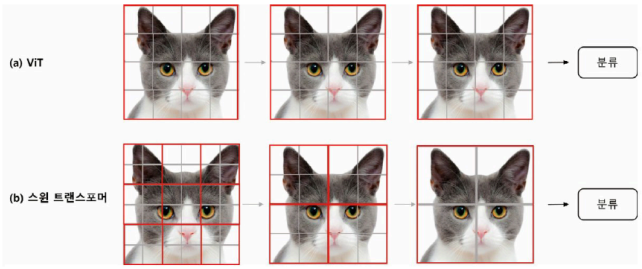


그림 10.6 ViT와 스윈 트랜스포머의 네트워크 구조

그림 요소	의미
회색 격자	패치
빨간색 격자	로컬 윈도우
(a) ViT	모든 계층에서 **전체 이미지 하나**에 어텐션, 패치 크기·개수 **동일**
(b) 스윈	로컬 윈도우로 계층마다 어텐션 패치 크기·개수가 **다양** (작은 윈도우 여러 개 → 점점 큰 윈도우)

- 로컬 윈도우 → 각 패치의 **세부 정보에 집중** / 시프트 윈도우 → **전역 특징** 확인 → 고해상도 이미지 효율 처리

> 🧠 ViT = 사진 전체를 처음부터 끝까지 한 장으로 보는 방식. 스윈 = 처음엔 돋보기로 작은 구역을 꼼꼼히 보고, 층이 올라갈수록 구역을 합쳐 점점 넓게 보는 방식 (CNN처럼 계층적).

표 10.4 ViT와 스윈 트랜스포머 비교

구분	ViT	스윈 트랜스포머
분류 헤드	[CLS] 토큰 사용	각 토큰의 **평균값** 사용
로컬 윈도우 사용	X	O
상대적 위치 편향 사용	X	O
계층별 패치 크기	동일함	다양함

3. 스윈 트랜스포머 모델 구조

- 이미지를 **패치 파티션**으로 구분 → **선형 임베딩(Linear Embedding)**, **스윈 트랜스포머 블록(Swin Transformer Block)**, **패치 병합(Patch Merging)** 연산을 반복

단계	하는 일
① 패치 분할 + 선형 임베딩	입력 이미지를 일정 크기 패치로 분할 → 각 패치를 **고정된 차원의 벡터**로 변환
② 스윈 트랜스포머 블록	어텐션 계층, 계층 정규화, 다층 퍼셉트론 등 포함 → 패치 간 상호작용
③ 패치 병합	분할된 패치들을 순차적으로 합치면서 이미지 전체 정보 추출 → 전체 이미지 분류

[그림 10.7 스윈 트랜스포머 구조]

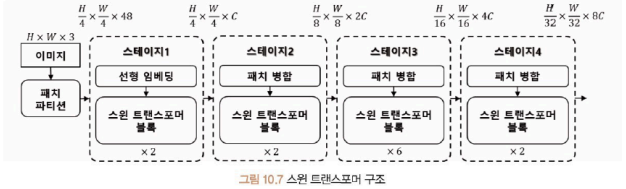


그림 10.7 스윈 트랜스포머 구조

구간	구성	블록 반복	출력 크기
입력	이미지 → 패치 파티션	–	H × W × 3 → H/4 × W/4 × 48
스테이지 1	**선형 임베딩** → 스윈 블록	×2	H/4 × W/4 × **C**
스테이지 2	**패치 병합** → 스윈 블록	×2	H/8 × W/8 × **2C**
스테이지 3	패치 병합 → 스윈 블록	×6	H/16 × W/16 × **4C**
스테이지 4	패치 병합 → 스윈 블록	×2	H/32 × W/32 × **8C**

- 기호 풀이: **H** = 이미지 높이, **W** = 너비, **C** = 채널(특징 개수). 스테이지가 올라갈 때마다 **가로·세로는 절반, 채널은 2배**
- 일반적으로 **4개 스테이지**, 각 스테이지는 서로 다른 해상도와 패치 크기
- 스테이지 1~4 공통 스윈 블록 = **두 가지 어텐션 모듈** (윈도우 격자 **안** 패치 관계 / 윈도우 격자 **간** 관계)

> 🔎 48은 어디서? 4×4 패치 하나에 픽셀 16개 × RGB 3채널 = **48**개 숫자. 패치 하나를 48차원 벡터로 편 것.
> 🧠 CNN에서 층이 깊어질수록 이미지 크기는 줄고 채널은 늘던 것(ResNet의 (28,28,64) → (14,14,128))과 똑같은 피라미드 구조.

4. 패치 파티션 / 패치 병합

패치 파티션 (Patch Partition)

- 입력 이미지를 **작은 사각형 패치로 분할**해 처리하는 방식
- 분할된 패치 → 트랜스포머 계층의 입력 / 공간 정보 보존 + 계산 효율성 향상

[그림 10.8 패치 파티션 구조]

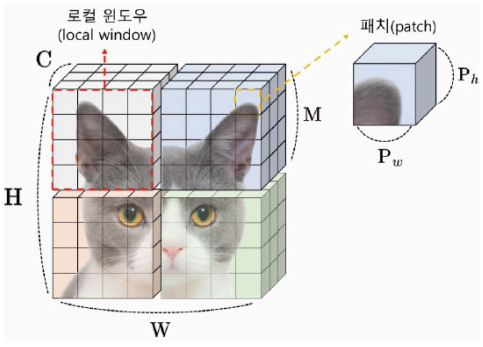


그림 10.8 패치 파티션 구조

기호	의미
C, H, W	이미지 텐서 [채널, 높이, 너비]
M	로컬 윈도우 크기 (윈도우 한 변에 패치 몇 개)
P_h, P_w	패치 하나의 가로·세로 크기
예	[3, 32, 32] 이미지 → P_h = P_w = **4**

패치 병합 (Patch Merging)

- 인접한 패치들의 정보를 **저차원으로 축소**하는 과정 → 모델 매개변수 수를 줄이기 위함
- 교재 설명: [C, H, W] → **[2C, H/2, W/2]**로 재정렬 후 2C 차원을 저차원으로 임베딩
  - 예: [3, 8, 8] → [6, 4, 4] → 3차원으로 임베딩 → [3, 4, 4]

5. 스윈 트랜스포머 블록

- 패치 파티션 이후 4개 스테이지 안에서 반복 적용, 선형 임베딩 또는 패치 병합 이후 수행
- 구성: **계층 정규화, W-MSA, MLP, SW-MSA**

[그림 10.9 스윈 트랜스포머 블록 구조]

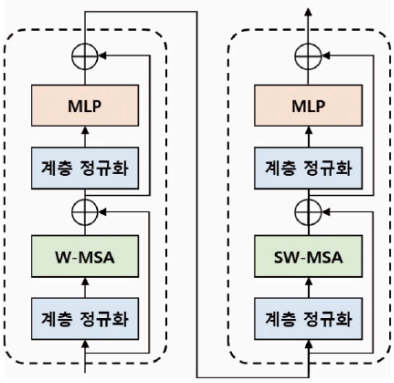


그림 10.9 스윈 트랜스포머 블록 구조

블록	순서
왼쪽 (짝수 번째)	계층 정규화 → **W-MSA** → ⊕ → 계층 정규화 → MLP → ⊕
오른쪽 (홀수 번째)	계층 정규화 → **SW-MSA** → ⊕ → 계층 정규화 → MLP → ⊕

- 기존 트랜스포머의 **MSA**(Multi-head Self-Attention) 대신 **W-MSA와 SW-MSA**를 **번갈아 순차적으로** 수행 → 그래서 블록 반복 수가 항상 짝수(×2, ×2, ×6, ×2)
- ⊕ = 잔차 연결 (ResNet의 숏컷처럼 입력을 다시 더함)

W-MSA vs SW-MSA

구분	W-MSA (Window MSA)	SW-MSA (Shifted Window MSA)
정의	로컬 윈도우를 사용하는 멀티 헤드 셀프 어텐션	윈도우를 가로/세로로 **이동한 후** 인접 윈도우 간 셀프 어텐션
방법	입력 특징 맵을 **중첩되지 않게** 나눔 → 각 윈도우에서 **독립적으로** 셀프 어텐션	윈도우 사이의 연결성 구축
장점	윈도우 내 지역 특징 분석, 계산 복잡도 **2차** → **1차**, 연산량 대폭 감소	윈도우 **간** 관계성 정보 추출
단점	**윈도우 내부**에서만 어텐션 → 윈도우끼리 정보 교환 X	윈도우 개수가 늘어 연산량 증가 → 순환 시프트 + 어텐션 마스크로 해결

[그림 10.10 W-MSA와 SW-MSA 어텐션 영역 비교]

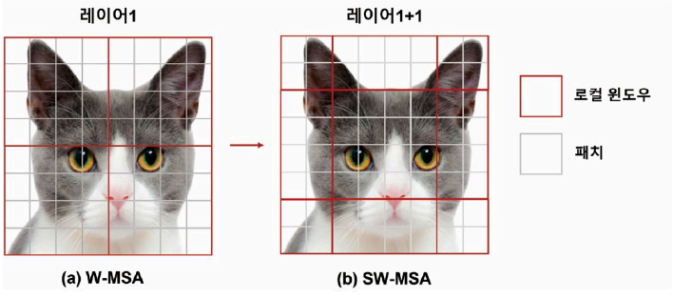


그림 10.10 W-MSA와 SW-MSA 어텐션 영역 비교

- (a) 레이어 l: 빨간 윈도우 4개(2×2)로 나눠 각자 어텐션
- (b) 레이어 l+1: 윈도우 경계를 옮김 → 이전 층에서 서로 다른 윈도우에 있던 패치들이 같은 윈도우에 들어감
- W-MSA는 윈도우가 이동하며 각 윈도우마다 연산 → 계산량이 많아 보이지만, **효율적인 배치 계산(Efficient Batch Computation)**으로 연산량 대폭 감소
  - 로컬 윈도우 연산에도 **배치 축**을 사용 → 연산 속도 향상 (윈도우들을 배치처럼 묶어 한 번에 계산)

6. 순환 시프트 & 어텐션 마스크

- SW-MSA는 W-MSA보다 로컬 윈도우 개수가 많아 더 많은 연산 필요
- 해결: **순환 시프트(Cyclic Shift) + 어텐션 마스크(Attention Mask)**

[그림 10.11 순환 시프트를 활용한 SW-MSA 어텐션 계산 과정]

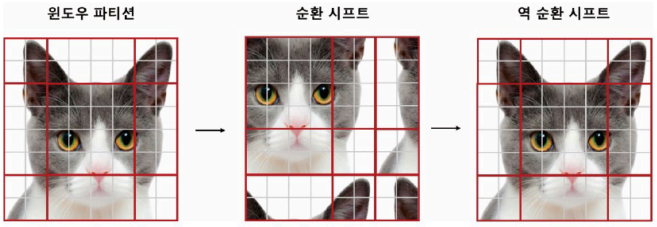


그림 10.11 순환 시프트를 활용한 SW-MSA 어텐션 계산 과정

단계	하는 일
① 윈도우 파티션	윈도우를 M/2만큼 옮기면 크기가 제각각인 윈도우 **9개**가 생김
② **순환 시프트**	이미지를 왼쪽 위로 **M/2만큼** 밀고, 밀려나간 부분을 반대편(오른쪽·아래)에 붙임 → 다시 **4개** 윈도우 모양
③ 어텐션 마스크	이어 붙인 조각들(원래 서로 멀리 있던 부분)끼리는 어텐션을 **못 하게 가림**
④ **역순환 시프트** (Reverse Cyclic Shift)	셀프 어텐션 후 원래 위치로 되돌림

- 순환 시프트 = 로컬 윈도우가 이동할 때 이동한 위치의 정보를 이전 위치에서 가져오는 것, 로컬 윈도우 사이즈 M보다 작은 **M/2**만큼 이동

[그림 10.12 순환 시프트 단계에서 마스크를 활용한 셀프 어텐션 구조]

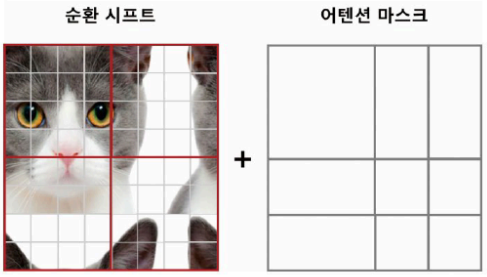


그림 10.12 순환 시프트 단계에서 마스크를 활용한 셀프 어텐션 구조

- 결과: 원래 **9개** 윈도우 파티션 → 9번 계산 필요 → 마스크 사용 시 **4개** 윈도우 파티션 · 4번 셀프 어텐션만으로 모든 로컬 윈도우 간 어텐션 계산
- 이동된 윈도우 정보는 보존하면서 불필요한 연산 감소

7. 상대적 위치 편향 (Relative Position Bias)

- 기존 ViT와 다른 점: 스윈의 셀프 어텐션은 **상대적 위치 편향**을 고려
- 목적: 로컬 윈도우 안 패치 간의 **상대적인 거리**를 임베딩

[그림 10.13 상대적 위치 편향 계산 과정]

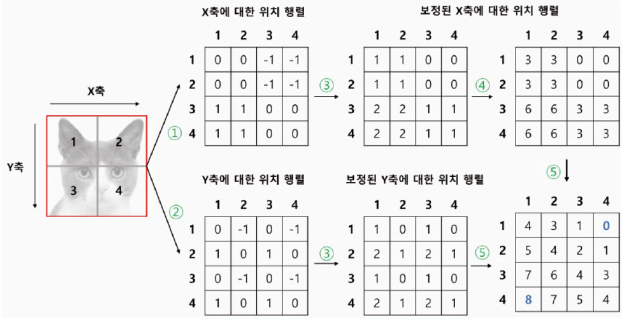


그림 10.13 상대적 위치 편향 계산 과정

- 예: 윈도우 크기 2 → 패치 1·2·3·4 (2×2)
- 패치 간 상대적 거리 = **X축**(가로 방향 거리) / **Y축**(세로 방향 거리) 두 방식으로 표기

단계	연산	결과
①	X축 위치 행렬: 같은 줄이면 0, 아래로 떨어지면 1, 위로 떨어지면 −1	값 −1~1
②	Y축 위치 행렬: 같은 줄이면 0, 오른쪽으로 떨어지면 −1, 왼쪽으로 떨어지면 1	값 −1~1
③	두 행렬에 **window_size − 1** (=1)을 더함	음수 없애기 → 값 0~2
④	X축 행렬에 **2 × window_size − 1** (=3)을 곱함	X와 Y가 겹치지 않게 자릿수 분리 → 0, 3, 6
⑤	X축 + Y축 행렬 더하기	**0~8** 사이 번호 (상대 위치 인덱스)

예제 10.10 상대적 위치 편향 계산 (①, ②)

코드	하는 일
`window_size = 2`	여러 이미지 패치를 포함하는 격자 크기 → 윈도우 내외부 패치 구분
`torch.arange(window_size)`	[0, 1] → `coords_h` , `coords_w`
`torch.meshgrid([coords_h, coords_w], indexing="ij")`	coords_h(i)·coords_w(j)로 **사각형 격자** 생성 → X좌표, Y좌표 배열 반환
`torch.stack`	(X, Y) 쌍 배열 생성
`torch.flatten(coords, 1)`	각 X·Y축 위치 인덱스 `coords_flatten`
`coords_flatten[:, :, None] - coords_flatten[:, None, :]`	모든 패치 쌍의 위치 차이 → ①, ② 결과

- 출력: `relative_coords` shape = **[2, 4, 4]** → (X·Y 2개 축, 패치 4개 × 패치 4개)

> 🔎 `[:, :, None] - [:, None, :]` = **브로드캐스팅** 트릭. 세로로 세운 목록과 가로로 눕힌 목록을 빼면, 모든 (i, j) 쌍의 차이가 4×4 표로 한 번에 나옴 (구구단표 만드는 원리).

예제 10.11 X, Y축에 대한 위치 행렬 (③, ④, ⑤)

코드	연산 단계
`x_coords = relative_coords[0, :, :]` / `y_coords = relative_coords[1, :, :]`	X, Y 분리
`x_coords += window_size - 1` , `y_coords += window_size - 1`	③
`x_coords *= 2 * window_size - 1`	④
`relative_position_index = x_coords + y_coords`	⑤ → 0~8 상대적 위치 좌표 행렬

예제 10.12 상대적 위치 좌표 → 벡터 변환

코드	의미
`num_heads = 1`	MSA 계산 시 헤드 수 (예제는 1개)
`relative_position_bias_table = torch.zeros((2*ws-1) * (2*ws-1), num_heads)`	헤드별 **0~8번, 9개**의 상대적 위치 편향을 가지는 테이블 (학습되는 값)
`table[relative_position_index.view(-1)]`	인덱스로 해당 위치의 편향 값 불러오기

코드	의미
`.view(ws*ws, ws*ws, -1)`	헤드별 i번째 패치 → j번째 패치 어텐션 값에 더해지는 텐서 **[4, 4, 1]**로 재구성

> 🧠 편향 테이블 = "상대 위치 → 가산점" 사전. 위치 관계가 9가지뿐이라 가산점도 9개만 배우면 되고, 패치 쌍마다 번호(0~8)로 사전을 찾아 점수를 가져옴.
> ⚠️ 교재 출력은 `torch.Size([1, 4, 4])`인데, 코드대로면 `[4, 4, 1]`이 나옴 (본문 설명도 [4, 4, 1]). 실제 스윈 구현에서는 여기에 `permute(2, 0, 1)`을 해서 **[헤드 수, 4, 4]** = [1, 4, 4] 형태로 바꿔 씀.

수식 10.2 상대적 위치 편향을 고려한 셀프 어텐션

수식 10.2 상대적 위치 편향을 고려한 셀프 어텐션

$$Attention(Q,K,V) = SoftMax(QK^T/\sqrt{d} + B)V$$

기호	의미
Q, K, V	쿼리, 키, 값 벡터
QK^T	쿼리와 키를 곱함 → 패치끼리 얼마나 관련 있는지 점수표 (^{T} = 전치, 행·열을 뒤집어 곱셈 가능하게)
√d	d = 벡터들의 임베딩 차원. 점수가 너무 커지지 않게 나눠줌
B	**상대적 위치 편향** (새로 추가된 부분!) — 거리 관계에 따른 가산점
SoftMax	점수를 합이 1인 비율로 바꿈
× V	그 비율대로 내용물(값)을 섞음

> 🧠 기존 어텐션 점수 = "내용이 얼마나 비슷한가". 여기에 B = "위치가 얼마나 가까운가(어느 방향인가)" 보너스를 더한 것.

8. 스윈 트랜스포머 모델 실습

- 앞의 ViT 실습 코드를 활용해 스윈 트랜스포머를 **미세 조정**
- 데이터세트 코드는 예제 10.1, 10.2, 10.3 그대로 사용

사용 모델 이름 해석: `microsoft/swin-tiny-patch4-window7-224`

부분	의미
`swin-tiny`	스윈 트랜스포머 가장 작은 버전 (Swin-T, 첫 채널 C = 96)
`patch4`	**4×4** 이미지 패치
`window7`	**7×7** 로컬 윈도우 (M = 7)
`224`	224×224 이미지
학습 데이터	**ImageNet-1k** (1,000개 클래스)

예제 10.13 사전 학습된 스윈 트랜스포머 모델

- `SwinForImageClassification.from_pretrained(...)` — 매개변수는 ViT와 동일 (`num_labels`, `id2label`, `label2id`, `ignore_mismatched_sizes=True`)
- `model.named_children()`을 중첩 for문으로 돌며 모듈 이름을 트리 형태로 출력

구조	하위 모듈	역할
swin	embeddings → patch_embeddings → projection `Conv2d(3, 96, kernel_size=(4, 4), stride=(4, 4))`	패치 파티션 + 선형 임베딩
	embeddings → norm, dropout	정규화, 드롭아웃
	encoder → layers → 0, 1, 2, 3	**4개 스테이지**
	layernorm	계층 정규화
	pooler	풀링 (토큰 평균)
classifier	–	클래스 구분

예제 10.14 패치 임베딩 모듈

- 패치 파티션·선형 임베딩 = `model.swin.embeddings.patch_embeddings`
- 반환값이 2개: `patch_emb_output, shape = ...` (출력 텐서, 출력 격자 크기)

단계	shape	의미
이미지	`[32, 3, 224, 224]`	배치 32
Conv2d (커널 4×4, 간격 4)	56 × 56 격자	(224 − 4)/4 + 1 = **56**
일렬로 나열	`[32, **3136**, **96**]`	56 × 56 = **3,136개** 패치, **96개** 채널

ViT vs 스윈 패치 임베딩 비교

구분	ViT	스윈 트랜스포머
패치 크기	16×16	**4×4** (훨씬 잘게)
패치 개수	196	**3,136**
채널	768	**96**
[CLS] 토큰	붙임 → 197개	**안 붙임**
분류 방식	[CLS] 벡터	토큰들의 **평균값**

- 왜 [CLS] 대신 평균? 객체 검출 같은 작업에서 입력 이미지에 **여러 물체**가 있으면 [CLS] 토큰 하나로는 모든 물체를 효과적으로 검출하기 어려움 → 평균값으로 성능 향상

예제 10.15 스윈 트랜스포머 블록

- 첫 번째 스테이지 확인: `model.swin.encoder.layers[0].named_children()`
- 출력

모듈	의미
blocks → 0	첫 번째 스윈 블록
layernorm_before	어텐션 전 계층 정규화
attention	(W-MSA / SW-MSA) 셀프 어텐션
drop_path	잔차 경로를 확률적으로 건너뛰는 정규화 기법

- 스윈 트랜스포머의 스테이지 = **blocks** + **downsample**

구성	의미
`blocks`	스윈 트랜스포머 블록 (blocks[0], blocks[1])
`downsample`	**패치 병합** (reduction + norm)

- 스윈 블록은 다시 **SwinLayer-0**과 **SwinLayer-1**로 구성

레이어	어텐션 방식
SwinLayer-0 (`blocks[0]`)	**W-MSA**
SwinLayer-1 (`blocks[1]`)	**SW-MSA**

10. SwinLayer 구조 (예제 10.16)

- `print(model.swin.encoder.layers[0].blocks[0])`
- **SwinLayer** = 스윈 트랜스포머의 기본 빌딩 블록이 되는 계층 = 스윈 트랜스포머 블록
- 순서 (그림 10.9): 계층 정규화 → W-MSA(또는 SW-MSA) → 계층 정규화 → MLP

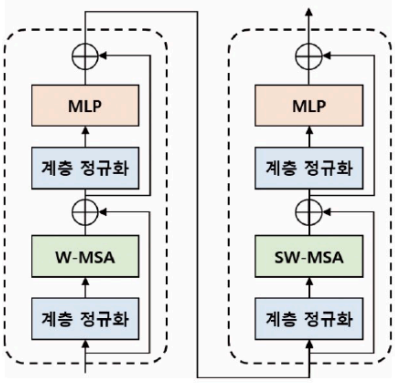


그림 10.9 스윈 트랜스포머 블록 구조

모듈	출력 내용	역할
`layernorm_before`	LayerNorm((96,))	**첫 번째** 계층 정규화
`attention` → self	query / key / value: `Linear(96 → 96)` 각각, dropout	**W-MSA (SW-MSA)** — Q·K·V 만들기
`attention` → output	dense `Linear(96 → 96)`, dropout	어텐션 결과 정리
`drop_path`	SwinDropPath(p=0.1)	잔차 경로를 10% 확률로 건너뜀 (과적합 방지)
`layernorm_after`	LayerNorm((96,))	**두 번째** 계층 정규화
`intermediate`	dense `Linear(96 → 384)` + **GELUActivation()**	MLP 앞부분 (차원 4배 확장)
`output`	dense `Linear(384 → 96)`, dropout	MLP 뒷부분 (원래 차원으로)

- **MLP** = 두 개의 연속된 선형 변환 계층 + **GELU** 활성화 함수
  - 입력 특징맵을 **더 깊은 차원(384)으로 매핑** → ReLU보다 더 안정적으로 학습

GELU (Gaussian Error Linear Unit)

- ReLU와 유사한 형태
- 입력값 **0 주변에서 부드럽게** 변화하는 특징

11. W-MSA, SW-MSA 모듈 실행 (예제 10.17)

코드	하는 일
`W_MSA = model.swin.encoder.layers[0].blocks[0]`	첫 스테이지 첫 블록 = W-MSA
`SW_MSA = ...blocks[1]`	두 번째 블록 = SW-MSA
`W_MSA(patch_emb_output, W_MSA.input_resolution)[0]`	패치 임베딩 + 입력 해상도(56, 56) 전달 → 출력 튜플의 0번째
`SW_MSA(W_MSA_output, SW_MSA.input_resolution)[0]`	W-MSA 결과를 SW-MSA에 이어서 전달

출력	shape
패치 임베딩 차원	`[32, 3136, 96]`
W-MSA 결과 차원	`[32, 3136, 96]`
SW-MSA 결과 차원	`[32, 3136, 96]`

- 입력·출력 차원이 **동일한 하이퍼파라미터**로 구성 → 블록을 몇 번 쌓아도 모양이 안 변함

12. 패치 병합 (예제 10.18)

- `patch_merge = model.swin.encoder.layers[0].downsample`

모듈	설정
reduction	`Linear(in_features=384, out_features=192, bias=False)`
norm	`LayerNorm((384,))`

- 출력: `patch_merge(SW_MSA_output, patch_merge.input_resolution)` → `[32, 784, 192]`

패치 병합 과정

단계	내용	shape
입력	56 × 56 = 3,136개 패치, 96채널	(N, 56, 56, 96)
① 4개로 나누기	가로·세로 한 칸씩 건너뛰며 4그룹으로 분할	(N, 28, 28, 96) × 4개
② 채널에 병합	4개 텐서를 채널 방향으로 이어 붙임 → 96 × 4 = **384**	(N, 28, 28, 384)
③ 정규화 + reduction	채널이 줄지 않았으므로 선형 변환으로 **절반(384 → 192)**	(N, 28 × 28 = **784**, **192**)

- 이런 방식으로 총 **4개 스테이지** 진행 → 스테이지마다 패치 크기·수 변화, 토큰 차원 **2배씩 증가**
- 마지막 스테이지 통과 시 `[N, 49(7×7), 768]` → 풀링 계층 → 클래스 예측

스테이지	패치 수	채널
1	56 × 56 = 3,136	96
2	28 × 28 = 784	192
3	14 × 14 = 196	384
4	7 × 7 = 49	768

> 🔎 Part 2에서 "패치 병합은 4C로 이어 붙인 뒤 2C로 줄인다"고 짚었던 게 바로 이 코드: 96 × 4 = 384 → Linear → 192.
> 🧠 2×2 타일 4장을 한 장으로 합치면 가로·세로 칸 수는 절반, 한 칸에 담긴 정보는 4배 → 너무 무거우니 2배로만 압축.
> 🔎 마지막 7 × 7 = 49칸 = 윈도우 크기(7×7)와 딱 맞음 → 마지막 스테이지에선 윈도우 하나가 이미지 전체를 덮어 **전역 어텐션**이 됨.

13. 스윈 트랜스포머 모델 학습 (예제 10.19)

- 예제 10.8 ViT 학습 코드에서 **변경 점만** 수정

변경 부분	ViT → 스윈
import	`SwinForImageClassification`
`model_init` 안의 모델	`SwinForImageClassification.from_pretrained("microsoft/swin-tiny-patch4-window7-224", ...)` (+ `ignore_mismatched_sizes=True`)
`image_processor`	`AutoImageProcessor.from_pretrained("microsoft/swin-tiny-patch4-window7-224")`
`output_dir`	`"../models/Swin-FashionMNIST"`
나머지 하이퍼파라미터	동일 (lr 1e-5, 배치 16, 3 에폭, f1 기준 ...)

- 학습 완료 → Swin-FashionMNIST 디렉터리에 체크포인트마다 학습 결과·성능 지표·하이퍼파라미터 저장

스윈 트랜스포머 학습 결과

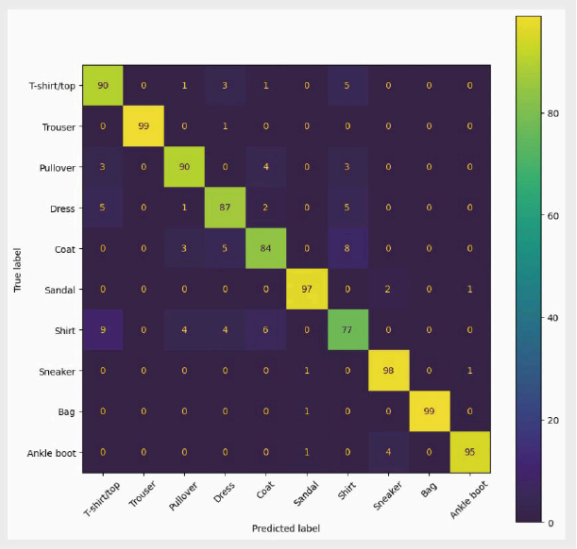
Epoch	Training Loss	Validation Loss	F1
1	0.3720	0.3046	0.8985
2	0.2579	0.2688	0.9138
3	0.2607	**0.2416**	**0.9159**

- 테스트 정확도 **약 91.6%**
- ViT와 비교: F1은 거의 동일, **검증 손실은 0.4357 → 0.2416으로 대폭 감소**

혼동 행렬 결과 (test 지표)

지표	값
test_loss	0.2417
test_f1	0.9160
test_runtime	4.3068초
test_samples_per_second	**58.048** (ViT의 약 2배)
test_steps_per_second	3.657

[스윈 트랜스포머 혼동 행렬]



클래스	ViT	스윈	변화
T-shirt/top	90	90	–
Trouser	99	99	–
Pullover	93	90	↓
Dress	90	87	↓
Coat	89	84	↓ (Shirt로 8개)
Sandal	97	97	–
Shirt	**72**	**77**	**↑ 개선**
Sneaker	98	98	–
Bag	100	99	–
Ankle boot	96	95	–

- 스윈이 **'셔츠'를 더 높은 확률로 예측** → 티셔츠·코트로 오분류하는 문제 완화
- 스윈 모델의 **수렴 속도가 더 빠름** (1 에폭부터 검증 손실이 낮음)

10.5 CvT (Convolutional Vision Transformer)

1. CvT란?

- 2021년 마이크로소프트 발표
- 합성곱 신경망 구조에서 활용하는 **계층형 구조(Hierarchical Architecture)**를 ViT에 적용한 모델

왜 필요할까?

모델	방식	한계 / 해결
ViT	모든 패치에 동일한 크기·간격 사용	이미지의 물체 크기·해상도를 계층적으로 학습하기 어려움
스윈	트랜스포머 블록마다 **다른 로컬 윈도우 크기**	계층적 특징 학습
CvT	CNN의 **계층적 구조**를 ViT에 적용	**지역 특징(Local Feature)** + **전역 특징(Global Feature)** 모두 활용, 비교적 적은 매개변수로 높은 성능

- 구조: **합성곱 계층으로 지역 특징 추출** + **ViT의 셀프 어텐션으로 전역 특징** 결합
- → 스윈 트랜스포머보다 **더 적은 매개변수**로도 비슷한 수준의 성능

2. CvT의 계층적 특징 구조

[그림 10.14 CvT 모델의 계층적 특징 구조 예시]

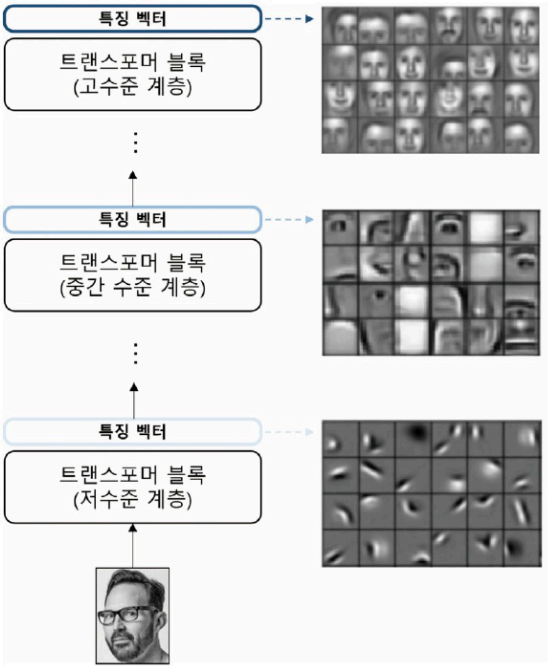


그림 10.14 CvT 모델의 계층적 특징 구조 예시

계층	얼굴 이미지 입력 시 표현하는 특징
저수준(Low-level) 트랜스포머 블록	선 또는 점
중간 수준(Mid-level)	눈이나 귀
고수준(High-level)	얼굴 구조

- → 계층형 구조로 물체의 크기나 해상도를 더 잘 표현

3. ViT vs CvT — 입력을 만드는 방식

구분	ViT	CvT
패치 처리	이미지를 패치 단위로 분할, **겹치지 않게** 처리 → 전체 정보 학습	처리 전에 **합성곱 연산으로 특징 맵 추출** → 유의미한 패턴 추출 후 일정 크기 패치로 나눠 입력, 패치들은 **겹쳐** 학습
임베딩	패치 임베딩 + 선형 임베딩	합성곱 연산으로 학습
위치 임베딩	필요	**없어도** 성능 저하 없이 다양한 해상도 이미지 처리 가능

표 10.4 ViT와 CvT 모델 비교

모델	위치 임베딩	패치 임베딩	어텐션 임베딩	계층형 구조
ViT	O	겹치지 않는 선형 임베딩	선형 임베딩	X
CvT	**X**	**겹치는 합성곱 임베딩 (overlapping)**	**합성곱 임베딩**	**O**

> 🔎 왜 위치 임베딩이 필요 없을까? 합성곱 필터는 "내 주변 이웃"을 보고 계산하기 때문에, 결과값 안에 이미 "어디쯤 있는지(주변이 어떤지)" 정보가 자연스럽게 녹아 있음. 번호표를 따로 붙일 필요가 없음.

4. 합성곱 토큰 임베딩 (Convolutional Token Embedding)

- 정의: **2D로 재구성된 토큰 맵**에서 **중첩 합성곱 연산**을 수행하는 임베딩
- 목적: ViT에 **계층적인 합성곱 신경망의 성질**을 추가
- (CvT의 "토큰 임베딩" = ViT의 "패치 임베딩"과 같은 뜻)

구분	기존 트랜스포머	CvT
방법	9개 이미지 패치 → **선형 임베딩**으로 Q·K·V 임베딩 → 셀프 어텐션	이미지 패치를 **2D 합성곱 임베딩**으로 Q·K·V 변환 → 셀프 어텐션

구분	기존 트랜스포머	CvT
2D 구조	일렬로 펴서 처리 → 근본적인 **2D 구조를 잃음**	2D 구조를 **보존**하면서 셀프 어텐션

📷 **[그림 10.15 2D 합성곱 기반의 어텐션 임베딩]**

단계	내용
①	패치 1~9 일렬 시퀀스
② **2D 구조 변형**	다시 3×3 격자로 세움
③ 합성곱	3×3 커널(K=3), 간격 1(S=1) → Q·K·V 각각에 합성곱 → 각 3×3
④ **1D 구조 변형**	다시 일렬로 펴서 기존 셀프 어텐션 방법으로 학습

- 효과
  - **지역 특징** 학습 가능
  - 시퀀스 길이를 줄이는 동시에 토큰 특징의 차원 증가 → CNN처럼 **다운 샘플링 + 특징 맵 수 증가**

> 🧠 일렬로 펴놓은 퍼즐을 다시 판 위에 깔아(2D) 이웃끼리 같이 보게 한 뒤(합성곱), 결과만 다시 줄 세워(1D) 어텐션에 넣는 것.

5. 어텐션에 대한 합성곱 임베딩 (Convolutional Projection for Attention)

- 정의: 2D로 재구성된 토큰 맵에서 **분리 가능한 합성곱 연산**을 적용해 성능 저하 및 계산 복잡도를 낮추는 연산
- 일반적으로 Q·K·V 임베딩은 차원을 동일하게 설정 → CvT는 매개변수 수를 줄이려고 **Q·K·V의 차원(개수)을 축소**
- = **2D 합성곱 기반의 축소된 어텐션 임베딩 (Squeezed Convolutional Projection)**

📷 **[그림 10.16 2D 합성곱 기반의 축소된 어텐션 임베딩]**

대상	커널(K)	간격(S)	생성 패치 수
쿼리	3×3	**1**	**9개**
키	3×3	**2**	**4개**
값	3×3	**2**	**4개**

[그림 10.17 축소된 셀프 어텐션 임베딩(열 × 행) 예시]

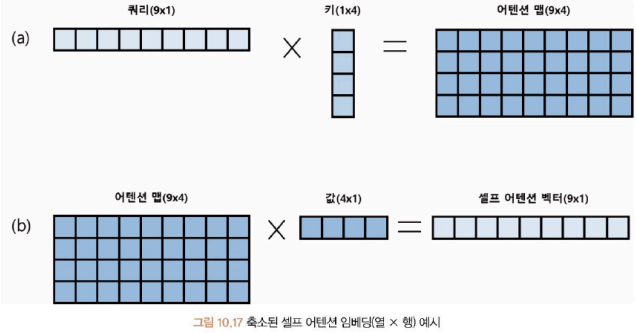


그림 10.17 축소된 셀프 어텐션 임베딩(열 × 행) 예시

단계	계산	결과
(a)	쿼리(9×1) × 키(1×4)	**어텐션 맵(9×4)** — 9개 쿼리 패치와 4개 키 패치 사이 중요도
(b)	어텐션 맵(9×4) × 값(4×1)	**셀프 어텐션 벡터(9×1)** — 값으로 가중치 반영

- 길이가 다른 쿼리(9), 키(4), 값(4)을 써도 결과 사이즈는 **9×1로 동일**
- → 입력과 출력 패치 길이가 같음 + 계산할 패치 개수는 줄어 **연산량 감소**

> 🔎 기호 풀이: "9×4"는 가로 9줄 × 세로 4칸짜리 표. 쿼리 9개 × 키 4개 = 36번만 비교하면 됨 (원래 9 × 9 = 81번). 결과 줄 수는 **쿼리 개수**가 정하기 때문에 키·값을 줄여도 출력 모양은 그대로.

층이 깊어질수록

- 트랜스포머 계층이 깊어질수록 **패치 간 관계를 학습하는 길이는 축소**, 대신 **벡터 차원 증가** → 매개변수 수 대폭 감소

- 예: 첫 블록 패치 64×64, 임베딩 128차원 → 마지막 블록 8×8, 512차원
- 즉 **이미지 패치 크기가 줄어든 만큼 임베딩 차원을 늘림** → 네트워크 안정성 향상, 모델 설계 단순화

6. CvT 모델 실습

- 사전 학습 모델: `microsoft/cvt-21` (ImageNet-1k, 224×224 이미지로 학습)
- 데이터세트 코드는 예제 10.1, 10.2, 10.3 활용

예제 10.20 CvT 이미지 데이터 전처리

부분	ViT · 스윈	CvT
image_processor	각 모델 이름	`"microsoft/cvt-21"`
Resize 크기	`size["height"]`, `size["width"]`	**`size["shortest_edge"]`** 두 번

- **shortest_edge** = 이미지의 너비나 높이 중 **더 작은 값**
- cvt-21은 모든 입력 이미지 크기를 **가장 짧은 이미지 길이로 정규화**해 사용 → 높이·너비 키 대신 shortest_edge 사용
- 나머지(`ToTensor`, `Lambda`로 3채널 복제, `Normalize`)는 동일

> 🔎 왜 키 이름이 다를까? 모델마다 이미지 프로세서 설정 파일이 달라서. ViT는 {"height": 224, "width": 224}, CvT는 {"shortest_edge": 224}로 저장돼 있음 → 없는 키를 쓰면 KeyError.

예제 10.21 사전 학습된 CvT 모델

- `CvtForImageClassification.from_pretrained("microsoft/cvt-21", num_labels, id2label, label2id, ignore_mismatched_sizes=True)`
- `named_children()` 중첩 for문으로 구조 출력

구조	하위 모듈
cvt	encoder → stages → **0, 1, 2** (3개 스테이지)
layernorm	계층 정규화
classifier	분류기

스윈과의 차이

구분	스윈	CvT
스테이지 수	4개	**3개**
분류기 전 풀링(pooler)	있음	**없음** — 합성곱 계층으로 이미 지역·전역 정보를 파악하므로
스테이지별 이미지 크기	–	**4배, 8배, 16배** 감소
임베딩 차원	–	반비례하게 **증가**

> ⚠️ 실제 Hugging Face의 `CvtForImageClassification`은 [CLS] 토큰(마지막 스테이지에서 추가)을 분류에 쓰고, 그게 없으면 토큰 평균을 냄. 교재의 "풀링 계층이 없다"는 건 **별도 pooler 모듈이 없다**는 뜻으로 이해하면 됨.

예제 10.22 CvT 모델의 스테이지 구조

- `stages = model.cvt.encoder.stages` → `print(stages[0])`
- 각 스테이지 = **CvtEmbeddings** + **CvtLayer**

모듈	설정	역할
CvtEmbeddings → projection	`Conv2d(3, 64, kernel_size=(7, 7), stride=(4, 4), padding=(2, 2))`	이미지를 패치 단위로 잘라 임베딩 (2D 합성곱)
CvtEmbeddings → normalization	`LayerNorm((64,))`	계층 정규화
CvtLayer → `convolution_projection_query`	`Conv2d(64, 64, k=3, **stride=1**, padding=1, **groups=64**)` + BatchNorm2d(64)	쿼리: 크기 유지
CvtLayer → `convolution_projection_key`	`Conv2d(64, 64, k=3, **stride=2**, padding=1, groups=64)` + BatchNorm2d(64)	키: **크기 절반**

모듈	설정	역할
CvtLayer → `convolution_projection_value`	`Conv2d(64, 64, k=3, **stride=2**, padding=1, groups=64)` + BatchNorm2d(64)	값: **크기 절반**
linear_projection	CvtSelfAttentionLinearProjection()	2D → 1D로 펴기

- 키·값의 간격을 1보다 큰 **2**로 설정 → 그림 10.16처럼 매개변수가 축소된 어텐션 임베딩

> 🔎 **groups=64** = 채널 64개를 64그룹으로 나눔 → 채널마다 **자기 필터 하나만** 따로 적용 (= **깊이별 합성곱, depthwise convolution**). 이게 앞에서 말한 "**분리 가능한 합성곱**"의 핵심. 채널끼리 섞지 않으니 계산량·매개변수가 크게 줄어듦.

첫 번째 스테이지 크기 계산

- 7×7 커널, 간격 4, 패딩 2

$$\frac{224 - 7 + 2 \times 2}{4} + 1 = 56.25 \;\rightarrow\; \text{버림} \;\rightarrow\; 56$$

- 기호 풀이: (이미지 길이 − 커널 + 양쪽 패딩) ÷ 간격 + 1. **패딩**(이미지 테두리에 0을 덧대는 것)이 2씩 양쪽이라 ×2
- 소수점은 버림 → 56 × 56 텐서, 출력 채널 64 → `[N, 64, 56, 56]`

> 🔎 커널 7 > 간격 4 → 필터가 움직일 때 **3칸씩 겹침** → 이게 표 10.4의 "**겹치는(overlapping)** 합성곱 임베딩". ViT(커널 = 간격 = 16)는 안 겹침.

예제 10.23 셀프 어텐션 적용

코드	하는 일	shape
`batch["pixel_values"]`	이미지	`[32, 3, 224, 224]`
`stages[0].embedding(...)`	합성곱 패치 임베딩	`[32, 64, 56, 56]`
`.view(batch_size, num_channels, height * width)`	가로·세로를 한 줄로	`[32, 64, 3136]`
`.permute(0, 2, 1)`	축 순서 바꾸기 (채널 ↔ 패치)	`[32, 3136, 64]` = 셀프 어텐션 입력
`stages[0].layers[0].attention.attention(hidden_state, height, width)`	CvtLayer의 셀프 어텐션	`[32, 3136, 64]` = 출력

- 합성곱 계층으로 계산된 패치 임베딩 → 가로·세로 크기로 재정렬해 CvtLayer에 입력
- **height, width를 같이 넘기는 이유**: 어텐션 안에서 다시 2D로 세워 합성곱(Q·K·V)을 해야 하니까
- 모듈의 입력 차원 = 출력 차원 (3,136개 이미지 패치 셀프 어텐션)

> 🔎 **permute** = 축 순서 바꾸기. [배치, 채널, 패치] → [배치, 패치, 채널]. 트랜스포머는 "패치(토큰) 목록, 각 토큰은 64차원 벡터" 모양을 원해서.

예제 10.24 CvT 모델 학습

변경 부분	값
import	`CvtForImageClassification`
`model_init` 안의 모델	`CvtForImageClassification.from_pretrained("microsoft/cvt-21", ...)`
`image_processor`	`AutoImageProcessor.from_pretrained("microsoft/cvt-21")`
`transform`	Resize에 `shortest_edge` 사용
`output_dir`	`"../models/CvT-FashionMNIST"`
나머지	동일

- 변경된 곳 = **사전 학습된 모델, 전처리 과정, 출력 디렉터리**

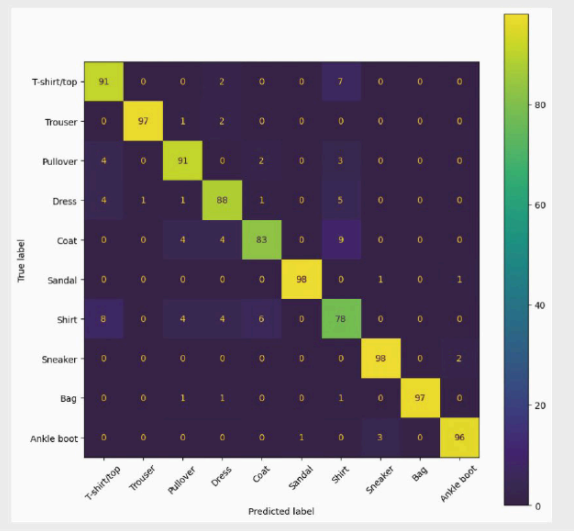
표 10.5 CvT 학습 결과

Epoch	Training Loss	Validation Loss	F1
1	0.7737	0.3269	0.8914
2	0.7008	0.2596	0.9172
3	0.6438	0.2460	**0.9173**

혼동 행렬 결과 (test 지표)

지표	값
test_loss	0.2460
test_f1	0.9174
test_runtime	5.583초
test_samples_per_second	44.778
test_steps_per_second	2.821

[CvT 혼동 행렬]



클래스	ViT	스윈	CvT
T-shirt/top	90	90	91
Trouser	99	99	97
Pullover	93	90	91
Dress	90	87	88
Coat	89	84	83
Sandal	97	97	98
Shirt	72	77	**78**
Sneaker	98	98	98
Bag	100	99	97
Ankle boot	96	95	96
합계 (정확도)	924 (92.4%)	916 (91.6%)	917 (91.7%)

- 교재: CvT 정확도 **91.3%**, 스윈 대비 F1 0.9159 → 0.9173으로 **소폭 개선**
- ViT 대비 '셔츠' 카테고리 성능 개선, 스윈과 비슷한 성능

10.6 비전 트랜스포머 모델 비교

표 10.6 비전 트랜스포머 모델 비교

구분	ViT	스윈 트랜스포머	CvT
모델 크기	327.325 MB	**105.227 MB**	120.791 MB
F1-점수	**0.9231**	0.9159	0.9173
초당 처리 수	29.636 eval/s	**58.048 eval/s**	44.778 eval/s

- FashionMNIST에서 비전 트랜스포머 계열 모델은 **작은 크기의 학습 데이터세트에서도** 높은 정확도와 우수한 성능

기준	1등
일반화 성능	**CvT** (교재 기준)
모델 크기 대비 성능	**스윈 트랜스포머** (가장 작고 가장 빠름)
F1-점수	**ViT**

- 결론: 비전 트랜스포머 모델은 분류 문제 해결에 매우 유용 → 풀어야 하는 문제와 사용처에 따라 적절히 선택

> 🧠 정리하면 — 정확도만 보면 ViT, 가볍고 빠르게는 스윈, 그 사이 균형은 CvT. 휴대폰이나 실시간 서비스처럼 자원이 적으면 스윈, 서버에서 정확도 최우선이면 ViT.
> ⚠️ 교재는 CvT의 "일반화 성능이 가장 좋다"고 했지만, 표만 보면 **검증 손실은 스윈(0.2416)이 CvT(0.2460)보다 근소하게 낮음**. 세 모델 차이가 1% 이내라 데이터가 작은(테스트 1,000장) 이번 실험만으로 우열을 단정하긴 어려움.